# Übungsblatt 8: KI-Beschleuniger & Edge AI

**Mikrocomputertechnik 3 (MCT3)**

## Zielsetzung

Sie vertiefen Vorlesung 8: Memory Wall bei neuronalen Netzen, Data-Level Parallelism / RVV, Systolic Arrays (NPUs) sowie Edge-AI-Praxis mit TensorFlow Lite Micro (Quantisierung, Arena, Inferenz).

**Bearbeitungszeit:** ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 12 P, Aufgabe 2: 12 P (Gesamt: 24 P)

## Teil 1: Theorie — Von der CPU zum KI-Beschleuniger

### Aufgabe 1.1: Die Memory Wall bei neuronalen Netzen

Neuronale Netze bestehen hardwareseitig primär aus großen Matrizen. Die grundlegendste Operation ist die MAC-Operation (Multiply-Accumulate).

1. Erläutern Sie die Struktur $a \leftarrow a + (w \times x)$ und die Bedeutung der Variablen.
2. Warum „verhungert“ eine klassische OoO-CPU an der *Memory Wall*? Argumentieren Sie mit der *Arithmetic Intensity* (Rechnen vs. Speicherzugriffe).
3. Warum helfen Branch Prediction und Reservation Stations bei Matrix-Multiplikationen kaum und verschwenden oft Energie?

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 1.2: Vektoren und die RISC-V Vector Extension (RVV)

1. Beschreiben Sie *Data-Level Parallelism* (DLP) im Kontrast zu ILP. Warum spart DLP Energie in Fetch/Decode?
2. RVV gilt als hardware-agnostisch. Wie ermöglicht `vsetvli`, dass derselbe Maschinencode auf schmaler Edge-CPU und breitem Server-Node läuft, ohne Neu-Kompilierung?

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 1.3: Systolic Arrays (NPUs)

1. Erklären Sie den Aufbau eines Systolic Arrays. Wie durchbricht es den Von-Neumann-Flaschenhals bezüglich RAM-Zugriffen?
2. Was ist *Weight-Stationary* Dataflow, und warum spart das Energie?
3. Nennen Sie einen entscheidenden Nachteil gegenüber einer General-Purpose-CPU.

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

## Teil 2: Praxis — Edge AI mit TensorFlow Lite Micro

**Szenario:** Ein trainiertes Modell (Schrittklassifikation) soll auf einem RISC-V-MCU laufen. Eingabe: 3-Achsen-Beschleunigung (Anknüpfung an I2C aus VL5).

### Aufgabe 2.1: Das Modell im Flash-Speicher

1. Warum quantisiert man Modelle für MCUs (Float32 → Int8)? Nennen Sie zwei Vorteile.
2. Das `.tflite`-Modell liegt als C-Byte-Array vor. Mit welchem Keyword und welchem Linker-Verhalten landet es im Flash statt im SRAM?

### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 2.1b: Modellgröße Float32 vs. Int8

Ein kleines Netz habe 250 000 Gewichte (Biases vernachlässigt).

1. Berechnen Sie den Speicherbedarf der Gewichte in Float32 (4 Byte/Gewicht) und in Int8 (1 Byte/Gewicht).
2. Um welchen Faktor schrumpft der Gewichtsspeicher durch Int8-Quantisierung?

### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 2.2: TFLM-Inferenzzyklus (C++)

Vervollständigen Sie das Snippet. Hilfsobjekte `resolver`, `error_reporter`, `input`, `output` und `read_i2c_sensor()` seien gegeben.

Hinweis: Das Snippet nutzt Float-Tensoren (`data.f`). Bei einem Int8-quantisierten Modell greifen Sie typisch auf `data.int8` zu und skalieren mit den Quantisierungsparametern (`scale`, `zero_point`) — hier didaktisch als Float belassen.

```
#include "tensorflow/lite/micro/all_ops_resolver.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
/* Modell-Array step_model_tflite ist inkludiert */

/* 1. Tensor Arena: 10 KiB, global */
/* TODO */

void setup_and_run_ai() {
    const tflite::Model *model = tflite::GetModel(step_model_tflite);

    tflite::MicroInterpreter interpreter(
        model, resolver, tensor_arena, 10 * 1024, error_reporter);

    /* 2. Arena vorbereiten */
    /* TODO: interpreter.... */

    while (1) {
        /* 3. Input: 128 Werte via read_i2c_sensor() */
        /* TODO */

        /* 4. Inferenz starten */
        /* TODO: interpreter.... */

        float score_laufen = output->data.f[2];
        /* 5. bei Wahrscheinlichkeit > 80 % melden */
        /* TODO: if (...) { printf(...); } */
    }
}
```

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 2.3: System-Auswirkungen in Renode

Sie führen das Programm in Renode auf einem RISC-V-SoC *ohne* NPU/RVV aus.

1. Was beobachten Sie bezüglich der Ausführungszeit von `Invoke()`?
2. Warum unterstreicht das die Notwendigkeit domänenspezifischer Architekturen am Edge?

**Ihre Lösung**

*(Notieren Sie Ihre Lösung hier.)*